Bahria University Islamabad, E-8 Campus

Department of Computer Science

Shooting Game with Score Management System

using 8086 Assembly Language

Course: Computer Organization and Assembly Language
Course Code: CEN 323
Instructor: Adnan Jelani

Group Members

Syed Huzaifa Farooq (01-135232-097)
Abdullah Javed (01-135232-003)
Saiyab Waheed (01-135232-085)

Submission Date: May 17, 2026

Contents

1 Abstract

This project is an interactive Shooting Game with a Score Management System developed using 8086 Assembly Language in EMU8086. The game allows a player to enter their name, shoot randomly generated targets, receive hit or miss feedback, and earn a score based on correct shots. The system stores player names and scores, displays a leaderboard, sorts scores in descending order, and provides grade-based feedback. The project demonstrates assembly language concepts such as arrays, loops, procedures, macros, stack usage, interrupts, arithmetic operations, conditional jumps, and indexed addressing.

2 Introduction

Assembly language provides a low-level understanding of computer architecture and hardware interaction. This project was developed to practically implement assembly concepts through an interactive shooting game. Instead of creating a simple arithmetic-based application, the project combines gameplay, animations, score management, sorting, and user interaction into one complete system.

3 Problem Statement

Most basic assembly projects only demonstrate simple loops or arithmetic operations. The goal of this project is to design a more advanced and interactive system that combines multiple modules including game logic, animations, score tracking, sorting, and leaderboard management in 8086 Assembly Language.

4 Objectives

- Develop an interactive shooting game in 8086 Assembly Language.
- Implement player name and score storage using arrays.
- Generate random targets using BIOS timer interrupt.
- Implement hit and miss detection using conditional jumps.
- Sort leaderboard scores using Bubble Sort.
- Use procedures and macros for modular programming.
- Demonstrate stack usage and register preservation.

5 Features

- Menu-driven interface
- Player name input
- Random target generation
- Hit and miss detection
- Score tracking
- ASCII-based animations
- Streak bonus system
- Grade calculation
- Leaderboard display
- Bubble sort ranking
- Sound effects using beep

6 System Design

The project consists of multiple modules:

- **Menu Module** - Handles menu navigation.
- **Game Module** - Controls gameplay and score updates.
- **Animation Module** - Displays gun, target, hit, and miss animations.
- **Score Module** - Stores scores and player names.
- **Leaderboard Module** - Displays sorted player rankings.
- **Sorting Module** - Uses Bubble Sort for descending order sorting.

6.1 Working Flow

1. User launches program.
2. Main menu is displayed.
3. Player enters name.

4. Random target is generated.
5. Player enters shot number.
6. Program checks hit or miss.
7. Score updates after every round.
8. Final score is stored.
9. Leaderboard displays sorted records.

7 Implementation Details

7.1 Arrays

Arrays are used to store names and scores.

- `scores db 10 dup(0)`
- `names db 200 dup(0)`

7.2 Macros

Macros used in the project:

- `NEWLINE`
- `BEEP`

7.3 Procedures

The project uses modular procedures such as:

- `draw_bar`
- `draw_gun`
- `draw_target`
- `anim_hit`
- `anim_miss`
- `show_countdown`
- `sort_scores`

7.4 Interrupts Used

- INT 21H for input/output operations
- INT 10H for screen handling
- INT 1AH for random target generation

7.5 Sorting Algorithm

Bubble Sort is used to sort scores in descending order. During swapping, corresponding player names are also swapped to maintain correct ranking.

8 Assembly Concepts Used

- Arrays
- Loops
- Procedures
- Macros
- Stack operations
- Conditional jumps
- Indexed addressing using SI and DI
- BIOS and DOS interrupts
- Arithmetic operations

9 CCA Mapping

9.1 CCA1: Complex Problem Solving

The project combines gameplay, animations, score management, sorting, and leaderboard systems into one integrated application.

9.2 CCA4: Design Constraints

The system is fully implemented in 8086 Assembly Language under EMU8086 limitations such as 16-bit registers and DOS interrupts.

9.3 CCA8: Team Collaboration

Different modules were divided among group members including gameplay, sorting, animations, testing, and documentation.

10 Testing and Results

Test Case	Expected Result	Actual Result
Play Game selected	Game starts	Successful
Correct target entered	HIT displayed	Successful
Wrong target entered	MISS displayed	Successful
Leaderboard opened	Sorted records shown	Successful
Exit selected	Program exits	Successful

11 Screenshots

Upload screenshots in Overleaf with these exact names:

- menu.png
- gameplay.png
- leaderboard.png

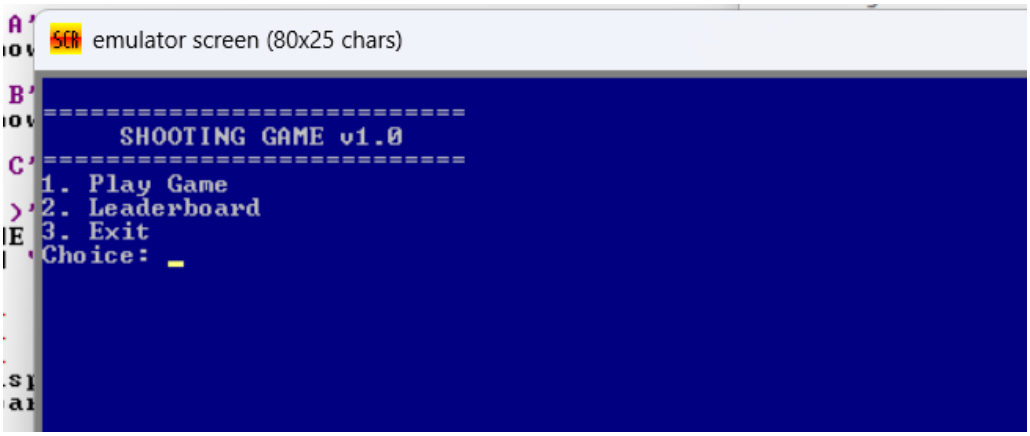

Figure 1: Main Menu Screen

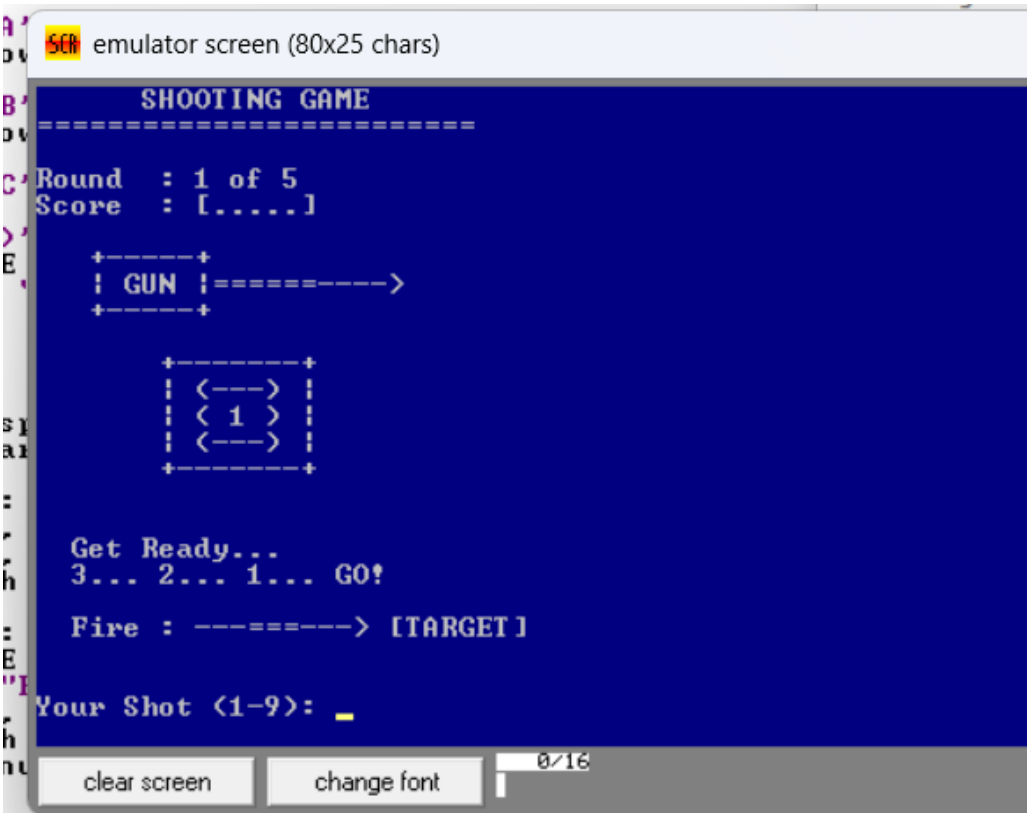

Figure 2: Gameplay Screen

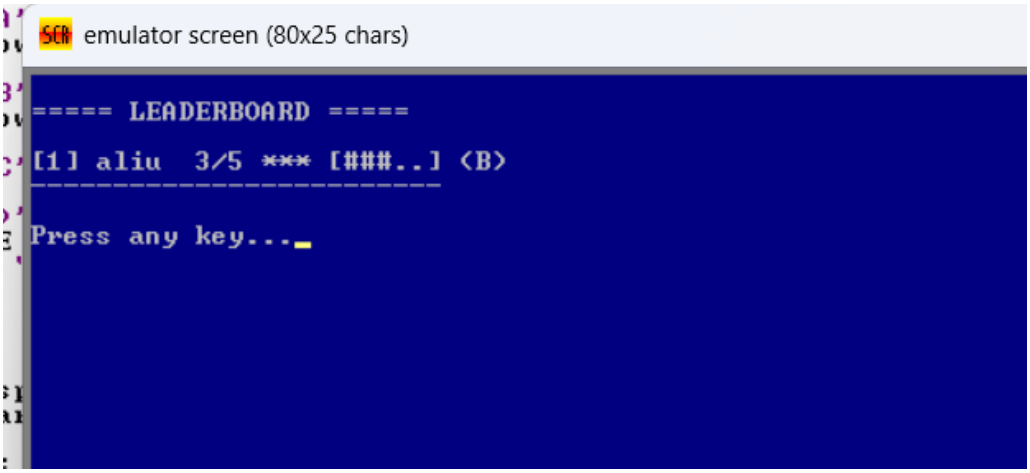

Figure 3: Leaderboard Screen

12 Team Contributions

Name	Registration No.	Module(s) Owned	Commit ID
Syed Huzaifa Farooq	01-135232-097	Game Logic and Testing	9c66cc2
Abdullah Javed	01-135232-003	Leaderboard and Sorting	8009cad
Saiyab Waheed	01-135232-085	UI, Animations, Documentation	5d5cb28

13 Challenges and Lessons Learned

During development, several challenges were faced including register conflicts, nested loops, and leaderboard sorting while keeping player names synchronized with scores. Multiple debugging and testing sessions were performed to improve program stability.

AI-assisted learning tools were consulted for syntax understanding, debugging guidance, and concept clarification, while the final implementation and integration were manually verified and tested.

14 Conclusion

This project successfully demonstrates how 8086 Assembly Language can be used to develop an interactive application with gameplay and score management features. The project combines multiple assembly language concepts including loops, arrays, procedures, macros, interrupts, sorting algorithms, and stack operations. It significantly improved understanding of low-level programming and hardware interaction.

15 References

- EMU8086 Official Documentation
- Intel 8086 Instruction Set Manual
- DOS Interrupt Reference for INT 21H, INT 10H, and INT 1AH
- Course Lab Manuals for Computer Organization and Assembly Language
- Stack Overflow discussions for assembly debugging
- GitHub repositories and open-source assembly examples

- AI-assisted learning tools including ChatGPT and Claude AI for debugging support and syntax understanding
- GitHub Repository:

`https://github.com/Saiyab-Waheed/Interactive-Shooting-Game-System`